ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Babtista Putri Grahani**
- **Chinese Name: 高俐雅**
- **Email: s115065432@m115.nthu.edu.tw / triababtista@gmail.com**
- **Student ID: 115065432**
- **University: 國立清華大學 National Tsing Hua University (NTHU)**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write “No” under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

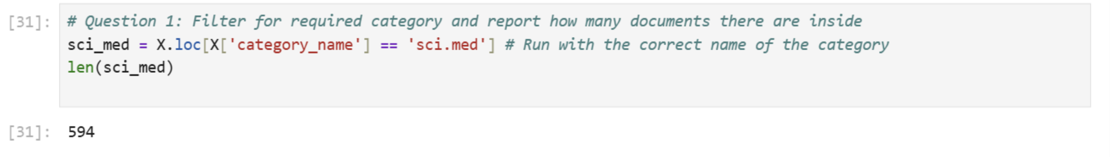
Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. There are 594 documents found in sci.med category.



```
[31]: # Question 1: Filter for required category and report how many documents there are inside
      sci_med = X.loc[X['category_name'] == 'sci.med'] # Run with the correct name of the category
      len(sci_med)

[31]: 594
```

2. Isolating a category is important for easier data exploration. Filtering categories helps us to know the data proportion across categories, whether their distribution is balanced or not. By looking at one category at a time, we can explore the domain knowledge/related terms within the category itself before finding patterns between categories or applying any algorithm. It will give us more context and examples of the specific data/category we want to explore. Therefore, we can choose the best data mining approach for the dataset.

AI used for this answer: No

If yes, how:

External references (links):

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: Dropping 40% of the data will remove almost half of your dataset, so it will not represent the original data anymore (biased data), and the result will be irrelevant. We will lose a lot of meaningful data/information that can be explored or analyzed.

To handle missing values for categorical data, we can predict the values using machine learning imputation methods, such as K-Nearest Neighbors (KNN), Random Forest (RF), and Sequential Hot-Deck (SHD). KNN method calculates the Euclidean distance of each row using the non-missing features, then it chooses the closest neighbors based on those distances. It calculates the weighted average of the features from its neighbors to fill in the missing values.

AI used for this answer: No

If yes, how:

External references (links):

https://doi.org/10.1016/j.imu.2023.101382

https://lnkd.in/p/gynMiFV4

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. The sample category proportions quite represent the original, but it's not perfect, because from the graph we can observe a bit of oversampling from comp.graphics category.
2. In an imbalanced dataset, simple random sampling will give the same distribution of overall data so it will also give imbalanced samples across categories. This might cause bias towards the majority-class. During data mining or classification process, the model only gets limited samples of the minority-class, leading to difficulty in discovering minority-class patterns (poor performance on minority-class detection) or overfitting of minority-class, because majority-class has more examples and more analyzed patterns in general. We can reduce this risk using under sampling or oversampling method to make the sample dataset balanced.

AI used for this answer: No

If yes, how:

External references (links): https://doi.org/10.1109/CASE49997.2022.9926548

https://simplicityissota.substack.com/p/balanced-samples-are-good-balancing

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

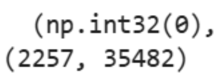
Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

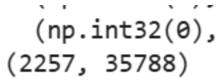
1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. Using stop_words='english' the shape of the document-term matrix is 35482. After applying stop_words=None, the columns of DTM increase by 306. It means that we put back 306 English stop words to the DTM.

   English stop words: 

   ```
   (np.int32(0),
   (2257, 35482)
   ```

   None: 

   ```
   (np.int32(0),
   (2257, 35788)
   ```

2. Because stop words are highly common in documents, the value of these words in DTM is most likely not zero. Stop words may add more columns of vocabulary, but most of their values will not be zero because they appear in most documents. However, it may also add some zero values to the matrix but it's not significant. As mentioned by Saif, et.al (2014), in their research on Twitter text sentiment analysis, general stop words removal does not really reduce sparsity significantly (except a specific stop list method, TF1), because rare terms are still there.

AI used for this answer: No

If yes, how:

External references (links): https://aclanthology.org/L14-1265/

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

1. As the variance threshold increases, the number of removed terms increases rapidly, leaving less kept terms (threshold 0.05 only keeps around 4% of the initial terms). A slight increase of threshold in a big dataset can remove more than half of the terms because hundreds of terms might have the same near-zero variance. The cut needs to be chosen carefully, so it will not remove important terms.
2. In a large dataset, it is not easy to find a word that has the same frequency throughout the entire dataset, so the removed words that have variance closest to 0 are mostly the words that only appear once through the entire dataset, the long tail of rare words. They are mostly meaningless words (e.g., zubkoff, zumder), typos, numbers, codes, people's names, or uncommon terms.
3. Near-zero variance features are either the words that always appear in every document or the words that only appear once or a few times throughout the entire documents (having near-constant values in DTM). Those terms will only add noise because they have no information for distinguishing any category from the others. Despite their large amount, the model will learn nothing from these features, so removing these irrelevant features not only reduces dimensionality, but also will make our dataset cleaner, leading to more accurate models.

Threshold	Number of terms kept (keep == True)
0.0	35482
0.01	5089
0.05	1491

AI used for this answer: No

If yes, how:

External references (links):

https://quantmemo.com/concepts/near-zero-variance-feature-removal

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

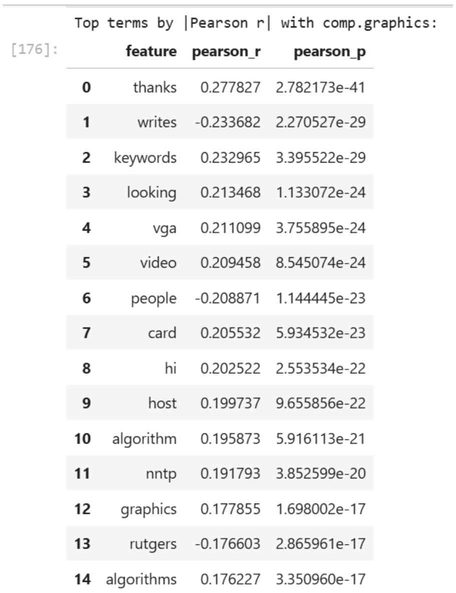
Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

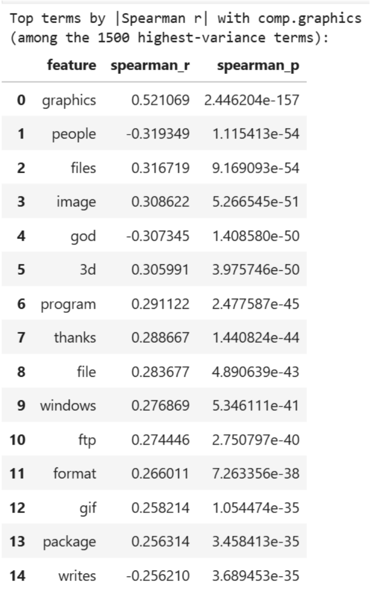
Answer:



```
Top terms by |Pearson r| with comp.graphics:
```

[176]:

	feature	pearson_r	pearson_p
0	thanks	0.277827	2.782173e-41
1	writes	-0.233682	2.270527e-29
2	keywords	0.232965	3.395522e-29
3	looking	0.213468	1.133072e-24
4	vga	0.211099	3.755895e-24
5	video	0.209458	8.545074e-24
6	people	-0.208871	1.144445e-23
7	card	0.205532	5.934532e-23
8	hi	0.202522	2.553534e-22
9	host	0.199737	9.655856e-22
10	algorithm	0.195873	5.916113e-21
11	nntp	0.191793	3.852599e-20
12	graphics	0.177855	1.698002e-17
13	rutgers	-0.176603	2.865961e-17
14	algorithms	0.176227	3.350960e-17



```
Top terms by |Spearman r| with comp.graphics
(among the 1500 highest-variance terms):
```

[177]:

	feature	spearman_r	spearman_p
0	graphics	0.521069	2.446204e-157
1	people	-0.319349	1.115413e-54
2	files	0.316719	9.169093e-54
3	image	0.308622	5.266545e-51
4	god	-0.307345	1.408580e-50
5	3d	0.305991	3.975746e-50
6	program	0.291122	2.477587e-45
7	thanks	0.288667	1.440824e-44
8	file	0.283677	4.890639e-43
9	windows	0.276869	5.346111e-41
10	ftp	0.274446	2.750797e-40
11	format	0.266011	7.263356e-38
12	gif	0.258214	1.054474e-35
13	package	0.256314	3.458413e-35
14	writes	-0.256210	3.689453e-35

1. Spearman and Pearson have noticeable different results and ranks, but they have some words in common (graphics, people, thanks). The top-correlated words are not generic stop words because it has been filtered out using stop_words = ‘english’. There are some domain-specific words like graphics, image, video, format, 3d, but there are also common terms used in general topics such as thanks, writes, people. Maybe around 50% of the top terms are domain specific.
2. The Pearson correlation measures the strength and direction of the **linear relationship** between two continuous variables. It assumes that the data are normally distributed and have linear relationship (increases/decreases by the same step). The Spearman rank correlation measures the strength and direction of the **monotonic relationship** between two ranked variables. Unlike Pearson, Spearman relationships can be linear or non-linear. If both variables generally increase or decrease together, it will result in a high Spearman coefficient. This method is suitable for ordinal/ranked data or data with outliers.

3. A word will have a higher Spearman rank correlation than Pearson correlation when the word appears many times only in particular documents inside the category but not in most documents, making it an extreme outlier. Suppose we want to measure the correlation between “graphics” and text length in the comp.graphics category. If “graphics” does not appear in a text, then appears once in a 2x longer text, then appears 15 times in a 3x longer text, “graphics” will have a higher Spearman rank correlation than Pearson correlation. It is because the increase rate of “graphics” is not consistent (not linear), but at least it increases as the text length increases. Spearman only looks at the order, so the outlier (15) does not affect it, while Pearson is pulled down by it.

AI used for this answer: Yes

If yes, how: Improving readability/phrasing of my own draft about higher Spearman-lower Pearson example

External references (links):

https://dkaarthick.medium.com/correlation-pearson-vs-spearman-rank-correlation-dc3e5420339b

https://support.minitab.com/en-us/minitab/help-and-how-to/statistics/basic-statistics/supporting-topics/correlation-and-covariance/a-comparison-of-the-pearson-and-spearman-correlation-methods/

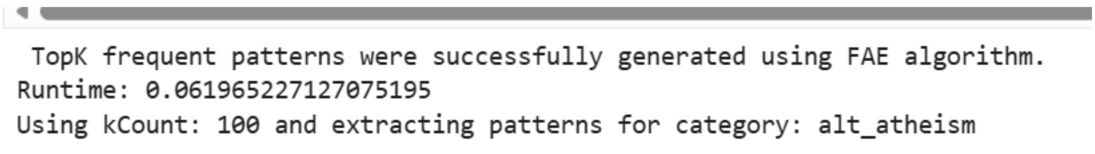
Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

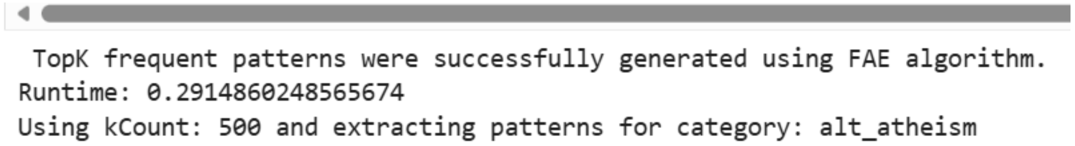
Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

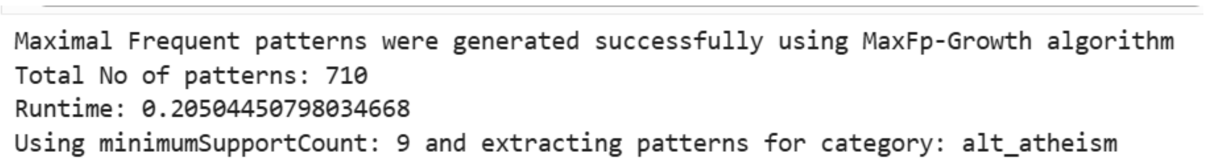


```
 TopK frequent patterns were successfully generated using FAE algorithm.
Runtime: 0.061965227127075195
Using kCount: 100 and extracting patterns for category: alt_atheism
```

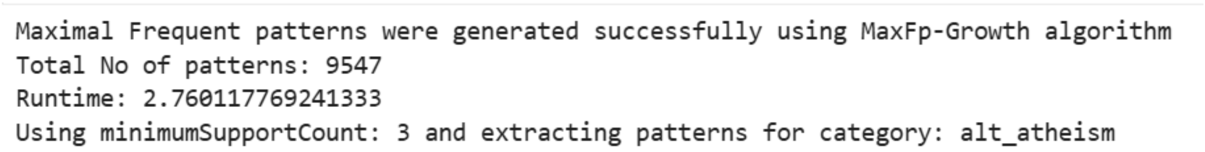


```
 TopK frequent patterns were successfully generated using FAE algorithm.
Runtime: 0.2914860248565674
Using kCount: 500 and extracting patterns for category: alt_atheism
```

Answer (1) — Runtime and pattern count when lowering minSup:



```
Maximal Frequent patterns were generated successfully using MaxFp-Growth algorithm
Total No of patterns: 710
Runtime: 0.20504450798034668
Using minimumSupportCount: 9 and extracting patterns for category: alt_atheism
```



```
Maximal Frequent patterns were generated successfully using MaxFp-Growth algorithm
Total No of patterns: 9547
Runtime: 2.760117769241333
Using minimumSupportCount: 3 and extracting patterns for category: alt_atheism
```

Decreasing MinSup will result in more discovered patterns and longer execution runtime, because there are more patterns passing the lower threshold (having minimum 3 supports instead of 9) and the algorithm needs to keep them all.

Both Top-K FAE and MaxFPGrowth need longer execution runtime to collect more patterns.

Answer (2) — Top-K k vs MaxFPGrowth minSup:'

In Top-K, k sets how many of the most frequent patterns to keep, regardless of their support. In MaxFPGrowth, minSup sets the minimum support a pattern needs to be kept, regardless of how many patterns result.

Answer (3) — Maximal vs standard frequent patterns:

Compared to standard frequent patterns, maximal frequent patterns only keep the longest patterns that pass the minimum support threshold. They do not list the smaller patterns inside a long pattern as separate patterns. Because only the largest patterns are kept, there is less redundancy, so the result set is smaller and more concise.

AI used for this answer: Yes

If yes, how: Improving readability and phrasing of my draft for number 2-3

External references (links):

https://doi.org/10.1007/s10489-020-02181-6

https://data-mining.philippe-fournier-viger.com/what-is-a-maximal-itemset/

https://www.geeksforgeeks.org/dsa/maximal-frequent-itemsets/

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. After text vectorization, a dataset can contain thousands of words as features, and each feature represents one dimension. A scatter plot can display only two or three dimensions, so the raw X_counts matrix cannot be plotted directly. Because we cannot draw a plot with thousands of dimensions, we must reduce the dimensions before visualizing the data.
2. We need to do **feature filtering** (or feature selection), for example using variance filtering or Spearman/Pearson filtering, to remove noise and to select important features for analysis. Then, we can apply dimensionality reduction methods such as PCA, UMAP, or t-SNE to transform the selected features into fewer dimensions, mitigating the curse of dimensionality.

AI used for this answer: Yes

If yes, how: Improving phrasing of my draft and finding supporting sources outside of reference from helper files.

External references (links):

https://webspace.science.uu.nl/~telea001/uploads/PAPERS/CiSE16/paper.pdf

https://www.ibm.com/think/topics/feature-selection

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. t-SNE and UMAP methods have more distinct and separated clusters that can be easily distinguished, while the PCA method showed heavy data overlapping around the center region.
2. PCA is a linear method, so it does not work well on non-linear data. In high-dimensional text data with many non-linear structures, the first two components may not have enough information to clearly separate the categories, so the categories overlap in the plot. t-SNE and UMAP are non-linear methods that focus on local structure, so similar documents stay close together and form separate clusters.
3. t-SNE is built to keep neighbors together. It penalizes placing near-neighbors far apart, but it does not care much about how far apart the different clusters are placed. Because it only optimizes for preserving local neighborhoods, the distances between different clusters do not represent true relationships in the original space. Two clusters that appear far apart are not necessarily more different than two clusters that appear closer together.

AI used for this answer: Yes

If yes, how: Explaining limitation of PCA and PCA as linear method, improving phrasing of my draft, and finding supporting sources

External references (links):

https://www.mv.helsinki.fi/home/mjxpirin/HDS_course/material/HDS10_nonlin.pdf

https://mbrenndoerfer.com/writing/tsne-dimensionality-reduction-visualization

https://mcpanalytics.ai/whitepapers/whitepaper-tsne

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. 4 binary columns. One column for each category.
2. The algorithm may assume raw-integer category label as ordinal data which has a hierarchical order or distance. The model may interpret the number wrongly, as if category 3 is bigger than 2 and 1, or category 2 is in between 1 and 3, which does not make sense. Category is nominal data, not ordinal.
3. One-hot binarization separates each category into distinct columns and assigns binary values, where 1 indicates the presence of a category and 0 indicates its absence. This ensures that there is no ranking or hierarchy among these categories, but rather only a value of true or false.

AI used for this answer: No

If yes, how:

External references (links):

https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/

https://www.digitalocean.com/community/tutorials/understanding-one-hot-encoding-in-machine-learning

---